Kubeflow + MLflow + KServe en entornos air-gapped: ¿reemplazo viable de HPE AI Essentials?

Informe de investigación — octubre 2026

Sí: Kubeflow, MLflow y KServe cubren casi todas las funciones de ML/LLMOps de HPE AI Essentials, porque HPE AI Essentials se construye sobre esas mismas herramientas open source. Lo que el stack open source no reemplaza es lo que HPE pone encima: hardware e integración validados de fábrica, soporte con SLA de un solo proveedor, NVIDIA AI Enterprise/NIM licenciado, el inferenciador propietario MLIS, la consola unificada y el ciclo de vida "llave en mano". En un entorno offline, todo eso pasa a ser trabajo de tu equipo de plataforma.

TL;DR

- **Qué es cada herramienta:** Kubeflow (Community Distribution 26.03.1, junio 2026; proyecto graduado en CNCF desde julio 2026) es la plataforma: notebooks, pipelines, entrenamiento distribuido, HPO, multi-tenancy y registro de modelos. MLflow (3.17.2, octubre 2026) es la capa de seguimiento de experimentos, registro de modelos, tracing y evaluación de GenAI. KServe (v0.20, agosto 2026) es la capa de serving en Kubernetes, con vLLM como runtime de primera clase y el CRD `LLMInferenceService` (todavía alpha) para LLMs.
- **Cómo se integran offline:** Kubeflow Pipelines orquesta, MLflow (sobre PostgreSQL y almacenamiento S3 local) registra, y KServe sirve leyendo `s3://…` desde ese mismo almacenamiento. KServe ya viene incluido en la distribución de Kubeflow. Para operar sin internet se necesitan un registro OCI privado (Harbor), espejos de PyPI/Helm/paquetes del SO, GPU Operator con imágenes locales y un IdP local (Keycloak/Dex). El repositorio de MinIO Community se archivó en GitHub el 25 de abril de 2026, así que conviene planear Ceph RGW o SeaweedFS.
- **¿Reemplaza a HPE AI Essentials?** Sí, si ya tienes o vas a comprar hardware no-HPE, si cuentas con al menos 2–3 ingenieros con dominio de Kubernetes que asuman el soporte y si no hay una exigencia contractual de SLA de un fabricante. No, si la organización necesita una solución llave en mano con soporte único, NIM certificados y poco personal de plataforma. Hay una opción intermedia con soporte comercial sobre los mismos componentes: Red Hat OpenShift AI o Charmed Kubeflow.

Hallazgos clave

1. **HPE AI Essentials es, en gran parte, un empaquetado curado de open source.** Según la Support Matrix oficial, AI Essentials 2026.07.1 trae Kubernetes 1.34.2, Kubeflow 1.11, MLflow 3.9.0, Ray 2.53.0, Airflow 3.1.7, Superset 5.0.0, Spark 3.5.5, EzPresto 0.293 y MLIS 2026.7.0. La versión 1.12.0 traía Kubeflow 1.10, MLflow 3.5.1 y Ray 2.48.0. La guía de administración de Private Cloud AI indica que Kubeflow incluye inte-

gración con KServe para el serving e inferencia, y que MLflow aporta el tracking y el registro centralizado.

2. **HPE va por detrás del upstream.** Kubeflow 1.11 frente a 26.03.1, y MLflow 3.9 frente a 3.17. Es normal en una plataforma validada, pero significa que pagas por estabilidad e integración, no por funciones nuevas.
3. **Lo propietario de HPE es concreto y acotado.** Incluye MLIS (HPE Machine Learning Inference Software, para desplegar modelos vLLM, NIM y Hugging Face desde registros S3/HF/NGC), el catálogo de modelos y los "Model Endpoints" con tokens de API, NIM preempaquetados de NVIDIA AI Enterprise, EzPresto (lakehouse federado), la consola unificada con SSO Keycloak y RBAC, la gestión vía GreenLake y la configuración air-gapped (solo en tamaños Medium y Large).
4. **El ecosistema open source maduró en 2025–2026.** Kubeflow se graduó en CNCF. MLflow 3 se reorientó hacia la observabilidad de GenAI (LoggedModel, tracing compatible con OpenTelemetry, AI Gateway con guardrails, control de acceso). KServe integró llm-d y vLLM y añadió canary en modo Standard, división de tráfico para LLMs, offloading de KV-cache y DRA para GPUs.
5. **Hay dos registros de modelos que se solapan.** El Model Registry de Kubeflow (rebautizado como "Kubeflow Hub" en 26.03.1) y el registro de MLflow son productos distintos. Recomiendo usar MLflow como fuente de verdad, y Kubeflow Hub solo si se necesita su catálogo de modelos o de MCP.

1. Kubeflow: propósito, componentes y estado a 2026

Propósito. Kubeflow es la base de herramientas para construir plataformas de IA sobre Kubernetes. Sus componentes se pueden desplegar por separado (como "subprojects") o como plataforma completa mediante la Kubeflow Community Distribution, antes llamada "AI Reference Platform". Entró a CNCF como incubating el 25 de julio de 2023 y pasó a **Graduated el 24 de julio de 2026**.

Versionado (cambio importante). La numeración pasó de 1.x a calendario:

- *Kubeflow AI Reference Platform 1.11*: 15 de diciembre de 2025.
- *Community Distribution 26.03*: 22 de marzo de 2026.
- *Community Distribution 26.03.1*: 15 de junio de 2026. Es la versión estable recomendada por kubeflow.org.

Según el README de kubeflow/community-distribution, el proyecto planea dos releases al año (por ejemplo 26.03 y 26.10, antes de cada KubeCon EU y NA) y da soporte comunitario *best-effort* a cada release durante 6 meses, con soporte comercial disponible aparte.

Componente	26.03	26.03.1
Kubeflow Pipelines (KFP)	2.16.0	2.16.1
Notebooks / Central Dashboard	1.10.0 / 1.10.0	Notebooks v1 actualizado + Workspaces (Notebooks v2) en beta; Dashboard v2
Katib (AutoML/HPO)	0.19.0	0.19.x
Trainer (sucesor del Training Operator)	2.1.0	2.2.0

Componente	26.03	26.03.1
KServe	0.16.0	0.18.0
Model Registry → “Kubeflow Hub”	0.3.7	0.3.9 (habilitado por defecto con UI y catálogo)
Spark Operator	2.5.0	2.5.0
Kubeflow SDK	0.3.0	—
Dependencias	Knative 1.21.1, OAuth2 Proxy 7.14.3	Istio 1.30.1, Knative 1.22.0, cert-manager 1.20.2, Dex 2.45.1, OAuth2-Proxy 7.15.2; CI sobre Kubernetes 1.36

Componentes clave:

- **Central Dashboard + Profiles/KFAM.** Multi-tenancy por namespace (un “Profile” por equipo o usuario) con Istio AuthorizationPolicies. La release 1.11 reforzó los valores por defecto multi-tenant y redujo el overhead por namespace.
- **Notebooks / Workspaces.** Servidores Jupyter, VS Code y RStudio dentro del clúster. Workspaces (v2) sigue en beta y no está pensado para producción.
- **Pipelines (KFP v2).** Orquestación de DAGs sobre Argo Workflows. Desde 26.03 es compatible con Pod Security Standards *restricted*.
- **Katib.** Ajuste de hiperparámetros y NAS. Hay una propuesta en curso para un CRD `OptimizationJob`.
- **Trainer v2.** API unificada y orientada a Python para entrenamiento distribuido (PyTorch, DeepSpeed, etc.) y fine-tuning de LLMs. Reemplaza al Training Operator v1.
- **Kubeflow SDK.** Desde 1.11 basta con `pip install kubeflow`. Integra entrenamiento, HPO, pipelines y Spark Connect en una sola API Python.
- **Kale 2.0.** Convierte notebooks anotados en pipelines de KFP v2.
- **KServe.** Viene **incluido** en la distribución como componente de serving (Serving Working Group). Knative solo hace falta si se usa el modo Knative de KServe. Para la próxima release, el control plane de KServe se mueve del namespace `kubeflow` al namespace `kserve`.

2. MLflow: propósito, versión y backends locales

Versión actual: 3.17.2, publicada en PyPI el 21 de octubre de 2026. La línea 3.x tuvo releases casi mensuales durante 2026: 3.9 en enero, 3.10 en febrero y marzo, 3.11.1 en abril, 3.12 en mayo, y luego 3.13, 3.15, 3.16 y 3.17.

Qué cubre hoy:

- **Tracking de experimentos.** Runs, parámetros, métricas y artefactos.
- **Model Registry.** Versiones, aliases como `@champion`, linaje.
- **Entidad `LoggedModel`.** Es de primera clase desde MLflow 3 y da un linaje fuerte entre modelos, runs, traces, prompts y métricas de evaluación.
- **GenAI/LLMOps:** tracing (con convenciones semánticas GenAI de OpenTelemetry desde 3.11.1); tracing multimodal (3.12); evaluación con jueces LLM y evaluación multi-turno (3.7); prompt registry; AI Gateway con guardrails (3.12) y controles de gasto (3.11.1); serialización sin pickle (3.11.1); control de acceso y ciclo de vida de trazas a escala (3.13).

- **Lo que se retiró en MLflow 3:** *MLflow Recipes* y los flavors fastai y mleap. *MLflow Projects* sigue existiendo, pero tiene poco peso en arquitecturas Kubernetes, donde KFP cumple ese rol.

Backends locales (patrón recomendado para air-gapped):

```
mlflow server \
  --backend-store-uri postgresql://mlflow:***@pg.interno:5432/mlflow \
  --artifacts-destination s3://mlflow-artifacts/ \
  --host 0.0.0.0 --port 5000

# Variables de entorno:
#   MLFLOW_S3_ENDPOINT_URL=https://s3.interno
#   AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY
```

- Con `--artifacts-destination`, el servidor actúa como **proxy de artefactos** (URI `mlflow-artifacts:/`). Los clientes no necesitan credenciales S3, lo cual es preferible en entornos regulados.
- SQLite es el backend por defecto desde 3.7, pero no sirve para uso multiusuario. En producción, usar PostgreSQL. MLflow 3.14+ permite además una réplica SQL de solo lectura.
- **Advertencia sobre MinIO:** la Community Edition dejó de publicar binarios e imágenes en octubre de 2025, entró en modo mantenimiento el 3 de diciembre de 2025 y su repositorio en GitHub se archivó el 25 de abril de 2026 (marcado como no mantenido). Sus CVEs futuros no se parchearán. Para S3 local nuevo conviene **Ceph RGW (vía Rook)** o **SeaweedFS** (Apache 2.0). Garage sirve para clústeres pequeños, aunque no soporta versionado. Otra opción es AIStor, de MinIO: "AIStor Free" es una edición standalone con licencia gratuita (un solo nodo) y "AIStor Enterprise" es la edición distribuida con soporte comercial.

3. KServe: serving en Kubernetes

Estado: proyecto CNCF *incubating*. La versión estable es **v0.20.0** (6 de agosto de 2026) y ya existe v0.21.0-rc0 (10 de septiembre de 2026). La rama master de Kubeflow ya apunta a KServe v0.21.0.

Modelo de recursos:

- **`InferenceService` (v1beta1).** Predictor, más transformer y explainer opcionales. Los runtimes se definen como `ServingRuntime` y `ClusterServingRuntime`: sklearn, XGBoost, LightGBM, PyTorch, TensorFlow, ONNX, Triton, MLServer (para MLflow), Hugging Face (con backend vLLM) y, desde v0.20, **vLLM como runtime independiente**.
- **`LLMInferenceService` (v1alpha1, desde v0.16).** Basado en llm-d. Ofrece enrutamiento consciente de prefijos y KV-cache, separación prefill/decode, multinodo (LeaderWorkerSet; sin Ray desde v0.18), autoscaling por utilización de KV-cache (WVA), APIs compatibles con OpenAI (incluida la Responses API, v0.18) y LoRA estático (v0.19).
- **Novedades de v0.20:** API de división de tráfico para despliegues progresivos de LLMs; **canary para InferenceService en modo Standard/RawDeployment**; offloading de KV-cache a memoria de CPU; API de tracing distribuido; DRA gestionado para GPUs;

montaje nativo de modelos como imágenes OCI (`oci+native://`); serving confidencial con TEEs; corrección del CVE-2026-48710 de Starlette.

Modos de despliegue:

Modo (nombre nuevo / antiguo)	**Dependencias**	**Autoscaling**	**Uso recomendado**
Standard / RawDeployment	Gateway API o Ingress; sin Knative	HPA o KEDA	Producción, GPUs y LLMs (sin *cold starts*)
Knative / Serverless	Knative Serving + Istio/Kourier	KPA, con scale-to-zero	Modelos predictivos con tráfico intermitente
ModelMesh	Componentes ModelMesh	—	Legado; no recomendado para trabajo nuevo

Inconsistencia en las fuentes. La página oficial "Configurations" de KServe sigue indicando Knative como modo por defecto, y la guía de instalación cambia el modo a Standard antes de instalar; en cambio, un tutorial sobre KServe 0.20 afirma que Standard ya es el modo por defecto. En la práctica conviene fijarlo explícitamente: `defaultDeploymentMode: Standard`. La guía de administración de KServe recomienda Standard para inferencia generativa. Además, según las notas de versión de OpenShift AI 3.5, Serverless (deprecado desde 2.25) y ModelMesh (deprecado desde 2.19) quedaron retirados, y las cargas que los usen deben migrarse antes de actualizar a 3.0.

GPUs y LLMs en la práctica: se piden con `nvidia.com/gpu` en los recursos. El `workerSpec` (tensor y pipeline parallel) permite multi-GPU y multinodo con vLLM. Para LLMs offline, lo correcto es cargar los pesos en S3 o PVC, o empaquetarlos como imagen OCI ("modelcar"), en lugar de usar `hf://`.

4. Integración entre las tres

Patrón de referencia:

```
[Kubeflow Notebooks/Workspaces]
        |
        v
[Kubeflow Pipelines (KFP v2)]
   prep -> Trainer/Katib -> eval
        |
        v
[MLflow Tracking + Registry]   (PostgreSQL + S3 local)
   alias @champion -> s3://.../artifacts/model
        |
        v
[KServe InferenceService / LLMInferenceService]
   storage-initializer lee del mismo S3
        |
        v
[Gateway/Istio] -> apps (Open WebUI, RAG, APIs)

Observabilidad: Prometheus/Grafana + DCGM exporter; MLflow tracing
```

Cómo sirve KServe un modelo MLflow:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: modelo-riesgo
  annotations:
    serving.kserve.io/deploymentMode: Standard
spec:
  predictor:
    model:
      modelFormat: { name: mlflow }
      protocolVersion: v2       # MLServer (mlserver_mlflow.MLflowRuntime)
      runtime: kserve-mlserver
      storageUri: s3://mlflow-artifacts/<exp_id>/models/<model_id>/artifacts
```

Puntos críticos:

1. **KServe no resuelve las URIs `models:/` ni `runs:/` de MLflow.** El paso de despliegue del pipeline tiene que traducir el alias del registro a la ruta física `s3://…` y aplicar un patch al `storageUri`. La documentación de MLflow lo indica explícitamente.
2. **El runtime MLServer necesita las dependencias del modelo.** En entornos offline hay que hornearlas en una imagen de runtime propia, o empaquetar el entorno (`environment.tar.gz`) y apuntar pip a tu mirror interno. Si no, falla la carga, un problema documentado en las discusiones del proyecto.
3. **En KServe ≥0.15, conviene fijar `runtime:` explícitamente** porque la autoselección por namespace no es fiable.
4. **Para LLMs no se usa el flavor MLflow.** Se registran los pesos (safetensors) y los metadatos en MLflow, y se sirven con el runtime vLLM o Hugging Face apuntando al mismo `s3://`. MLflow aporta el tracing y la evaluación de la aplicación GenAI.

Kubeflow Model Registry (“Hub”) frente a MLflow Registry:

- El registro de Kubeflow nació como contribución principal de Red Hat en Kubeflow 1.9. Es un catálogo de metadatos de modelos, datasets y métricas, integrado con Pipelines y KServe.
- En 26.03.1 pasó a llamarse **Kubeflow Hub** e incluye Model Catalog y MCP Catalog.
- MLflow es más maduro en tracking y linaje de experimentos y en GenAI.
- **Recomendación:** MLflow como fuente de verdad. Activar Hub solo si se quiere su catálogo en el dashboard de Kubeflow, y no mantener dos registros de versiones en paralelo.
- Red Hat converge en la misma dirección: OpenShift AI 3.4/3.5 incluye un **MLflow Operator** (CRDs `MLflow`/`MLflowConfig`, grupo `mlflow.kubeflow.org`). Cada proyecto de OpenShift se mapea 1:1 a un *workspace* de MLflow, con autorización por RBAC de Kubernetes (SelfSubjectAccessReview). Es justo la multi-tenancy que le falta a un MLflow “vanilla”.

Ejemplo de flujo end-to-end (clasificador tabular):

1. El científico de datos explora en un Notebook de su Profile.
2. Con Kale o el SDK se compila un pipeline KFP con los pasos: ingesta → features → `TrainJob` (Trainer) o Katib → `mlflow.log_model` → evaluación con `mlflow.evaluate`.
3. Si las métricas superan el umbral, el pipeline registra la versión y mueve el alias `@champion`.
4. Un paso final crea o actualiza el `InferenceService`: canary al 10 % y luego promoción.

5. Prometheus y Grafana monitorean latencia y GPU, y para LLMs las trazas van a MLflow.

5. Despliegue offline/air-gapped

Artefactos que hay que espejar:

Artefacto	Herramienta sugerida	Notas
Imágenes de contenedor (Kubeflow, KServe, Istio, Knative, cert-manager, Dex/OAuth2-proxy, runtimes vLLM/Triton/MLServer, GPU Operator)	**Harbor** (con escaneo Trivy y firma cosign), cargado con `skopeo copy --all` u `oc-mirror`	Copiar índices **multi-arquitectura** (`--all`) si hay nodos ARM64
Charts Helm	Harbor (OCI)	KServe distribuye charts OCI; Kubeflow tiene charts Helm experimentales y kustomize como vía oficial
Paquetes Python	devpi, Nexus o bandersnatch	Notebooks, componentes KFP, entornos MLServer y MLflow
Paquetes del SO / drivers	apt-mirror o reposync	El GPU Operator los necesita para compilar el driver, salvo que se usen **contenedores de driver precompilados**
Pesos de modelos	Almacenamiento S3 local o imágenes OCI de modelo	`HF_HUB_OFFLINE=1`; evitar `hf://` y `nvcr.io` en tiempo de ejecución
Identidad	Keycloak (federado a AD/LDAP) → Dex/OAuth2-proxy	Kubeflow usa Dex + OAuth2-proxy por defecto
Certificados	cert-manager + CA interna	KServe e Istio dependen de cert-manager

GPU Operator offline: NVIDIA documenta el escenario “Full Air-Gapped (w/o HTTP Proxy)”. Los nodos tiran de un registro local, el contenedor de driver usa un repositorio de paquetes local, y en `values.yaml` se reescribe `repository:` de cada imagen (operator, driver, toolkit, device-plugin, dcgm-exporter, MIG manager, validator). Con drivers precompilados no hace falta el mirror de paquetes.

Distribuciones de Kubernetes:

Opción	Air-gap	Soporte comercial	Comentario
RKE2 + Rancher	Nativo (tarballs de imágenes y registro privado)	SUSE	La mejor relación control/esfuerzo para un DIY; perfil CIS
k3s	Nativo	SUSE	Bueno para laboratorio o edge (p. ej. un nodo DGX Spark/GX10); no recomendado para producción multi-GPU
OpenShift + OpenShift AI	Maduro (`oc-mirror`, registro espejo)	Red Hat	La alternativa “open source con SLA” más completa; requiere OCP ≥4.19.9 para RHOAI 3.3

Opción	Air-gap	Soporte comercial	Comentario
Charmed Kubeflow (Canonical)	Sí (con Juju/Terraform)	Canonical	1.11 publicado el 13 de abril de 2026, validado en K8s 1.32–1.34; 1.11-ubuntu2 el 9 de septiembre de 2026 (con Ambient Mesh)
deployKF	Posible	No	Proyecto de un solo mantenedor que no aparece en la lista de distribuciones mantenidas de kubeflow.org; no recomendado para un proyecto nuevo
Otras de la lista oficial	—	prokube.ai, Nutanix, Azure, QBO	Kubeflow.org aclara que **no certifica** ninguna distribución

Desafíos y buenas prácticas:

- **Inventario reproducible de imágenes.** Generar la lista desde los manifests (`kustomize build | grep image:`) para cada release y versionar el "bundle" offline. Cada actualización implica re-espejar decenas de imágenes.
- **Compatibilidad de versiones.** Las versiones de Istio, Knative, cert-manager y Dex validadas por Kubeflow son estrictas. Las actualizaciones tienen cambios incompatibles: Dashboard 2.0 obliga a limpiar recursos antes de aplicar, y KServe 0.16→0.17 reestructuró los RoleBindings de LLMISvc y dividió el chart Helm en diez.
- **Arquitectura ARM64.** El DGX Spark y el ASUS GX10 son ARM64 (Grace Blackwell GB10). Muchas imágenes del ecosistema Kubeflow se publican sobre todo para amd64, y MLflow tuvo que hacer excepciones para imágenes ARM (3.15.1). Hay que verificar el soporte multi-arch imagen por imagen. Recomendación: usar esos equipos como estaciones de desarrollo o clúster de laboratorio, y la plataforma productiva en x86 + GPUs de datacenter.
- **GitOps (Argo CD) dentro de la burbuja.** El repositorio Git interno es la fuente de verdad. Políticas con Kyverno o Gatekeeper, y backups con Velero de PostgreSQL y de los buckets.
- **Seguridad de la cadena de suministro.** Escaneo en Harbor, SBOMs, y firma y verificación de imágenes antes de cruzar al entorno aislado.

6. HPE AI Essentials / HPE Private Cloud AI

Qué es. Es la capa de software de **HPE Private Cloud AI (PCAI)**, la "AI factory" llave en mano co-diseñada con NVIDIA. Combina hardware HPE (ProLiant DL380a Gen12 con RTX PRO 6000 Blackwell, entre otros), almacenamiento y red, más NVIDIA AI Enterprise y AI Essentials, todo gestionado vía GreenLake.

Componentes open source que integra (verificados en la documentación oficial y la Support Matrix):

- Kubeflow, con Notebooks, Pipelines, Katib y KServe;
- MLflow, con RBAC sobre experimentos y whylogs;
- Ray, con Ray Serve, Ray Tune y vLLM sobre Ray Serve;

- Spark Operator y Livy, Airflow, Superset, Feast, JupyterHub y VS Code;
- Prometheus para métricas, Perses para dashboards y Keycloak como proveedor OIDC local para SSO.

Capas propietarias o de valor agregado:

- **MLIS.** Despliegue y escalado de modelos con registros S3, Hugging Face y NGC, y formatos vLLM, NIM y custom. Ofrece despliegue multinodo de vLLM e integración con el catálogo de modelos y "Model Endpoints" con tokens de API.
- **NVIDIA AI Enterprise / NIM preempaquetados.** Por ejemplo Llama 3 8B/70B y embeddings y rerankers de Retrieval QA, además de los blueprints AI-Q y Omniverse.
- **EzPresto.** Lakehouse federado con conectores a Hive, Delta, Oracle, Snowflake, Teradata y otros, y PrestoMCP desde 1.12.
- **Consola unificada.** Despliegue GenAI "en 3 clics", gestión de usuarios y recursos, integración con GreenLake for File Storage y Data Fabric, observabilidad con OpsRamp y actualizaciones "one-click".
- **Air-gapped.** En marzo de 2026, en NVIDIA GTC San José, HPE anunció que el sistema PCAI grande ya estaba disponible en configuración air-gapped. Las QuickSpecs (julio de 2026) la limitan a los tamaños Medium y Large; no aplica al Developer System ni al Small. Existe documentación air-gapped específica de AI Essentials 1.12.x, incluida la aplicación de la licencia sin conexión. La activación de NVIDIA AI Enterprise en air-gapped es manual, con seriales de GPU leídos desde iLO y registrados en NGC. Los blueprints que requieren conectividad externa no están soportados en el modo aislado.

Licenciamiento y costos:

- AI Essentials se vende como E-LTU de 1, 3 o 5 años (SKUs S6E23AAE, S6E24AAE y S6E25AAE) **sin precio público**: el HPE Store muestra "solicitar cotización".
- Por debajo, la activación consume licencias **Ezmeral vCPU y vGPU** desde My HPE Software Center. NVIDIA AI Enterprise se licencia por GPU.
- PCAI se vende de forma tradicional (capex) o como suscripción GreenLake, y el soporte va incluido en la suscripción.
- Para Colombia el precio depende del canal o partner, así que hay que pedir cotización formal desglosada: hardware, AIE, NVAIE y servicios.

7. Comparación: HPE AI Essentials frente al stack open source

Capacidad	**HPE AI Essentials (PCAI)**	**Stack OSS (Kubeflow + MLflow + KServe + complementos)**	**Brecha OSS**
Notebooks / IDE	Kubeflow Notebooks, JupyterHub, VS Code	Kubeflow Notebooks/ Workspaces	Ninguna
Orquestación	KFP + Airflow	KFP (+ Airflow opcional)	Ninguna
Entrenamiento distribuido / HPO	Kubeflow, Katib, Ray	Trainer v2, Katib, KubeRay	Ninguna; upstream es más nuevo
Tracking / registro	MLflow 3.9	MLflow 3.17 + (Kubeflow Hub opcional)	Ninguna en funciones; falta multi-tenancy de MLflow (proxy/OIDC u operator de Red Hat)

Capacidad	HPE AI Essentials (PCAI)	Stack OSS (Kubeflow + MLflow + KServe + complementos)	Brecha OSS
Serving predictivo	KServe	KServe	Ninguna
Serving LLM	MLIS (vLLM/NIM), KServe, Ray Serve	KServe + vLLM / LLMInferenceService	Construir UX, catálogo y gestión de tokens (LiteLLM/Envoy AI Gateway + Keycloak)
NIM / NVAIE	Preempaquetado, con soporte	Opcional, licenciando NVAIE aparte	Costo y licencia aparte
Datos / lakehouse	EzPresto, Spark, Feast, Superset	Trino, Spark Operator, Feast, Superset	Integración manual
Almacenamiento S3	Integrado (GreenLake File, Data Fabric)	Ceph RGW / SeaweedFS	Operar el almacenamiento uno mismo
Identidad / SSO / RBAC	Keycloak + RBAC unificado	Keycloak + Dex + Profiles + RBAC K8s	Configurar y mantener el SSO entre herramientas
Observabilidad	Prometheus, Perses, OpsRamp	Prometheus/Grafana + DCGM + MLflow tracing	Dashboards y alertas propios
Air-gapped	Producto soportado (Medium/Large)	Factible; pipeline offline DIY	**Grande**: bundles, re-especado y pruebas en cada upgrade
Ciclo de vida / upgrades	One-click, validado	Manual; cambios incompatibles frecuentes	**Grande**
Soporte / SLA	HPE + NVIDIA, un solo responsable	Comunitario best-effort (6 meses por release)	**Grande** (mitigable con Red Hat, Canonical o SUSE)
Cumplimiento	Hardening y posicionamiento soberano	Lo que construyas y audites	Media/Grande
Costo de licencia	Cotización (no pública)	$0 en software	A favor del OSS
Esfuerzo de ingeniería	Bajo/medio	Alto: estimo 2–3 FTE de plataforma	A favor de HPE

Lectura. En funciones, el OSS **iguala o supera** a AI Essentials, porque corre versiones más nuevas. Las brechas reales son **operativas y contractuales**: soporte, upgrades validados, el pipeline air-gapped empaquetado, la UX unificada y el inferenciador gestionado. El costo total de propiedad depende sobre todo de ese esfuerzo humano. Sin precios públicos de HPE no se puede dar un cálculo cerrado. La comparación correcta es *licencia AIE + NVAIE* contra *2–3 FTE de plataforma más el soporte comercial opcional*.

Condiciones en que el OSS es un reemplazo viable:

1. El hardware no es HPE PCAI, o ya existe.
2. Hay al menos 2–3 ingenieros con experiencia real en Kubernetes, Istio y GPU.
3. Se aceptan soporte comunitario o un tercero (Red Hat, Canonical, SUSE) en lugar del fabricante del hardware.
4. Los casos de uso son vLLM o modelos open-weights, sin dependencia de NIM.
5. Hay tolerancia a ciclos de actualización semestrales planificados.

Cuándo es mejor HPE (u otro llave en mano): equipo de plataforma pequeño; necesidad contractual de SLA de extremo a extremo; compra de hardware nueva; requisitos de NIM o NVAIE certificados; prioridad de tiempo a producción en semanas.

8. Alternativas de referencia

- **Red Hat OpenShift AI (3.x).** Se basa en Kubeflow (notebooks, pipelines, Training Operator/Trainer, Model Registry), KServe, Ray, Kueue, TrustyAI, Feast, Llama Stack y ahora el MLflow Operator. Tiene un instalador desconectado documentado (`oc-mirror`) y un chart Helm OCI para GitOps (agosto de 2026). Requisitos: S3 para pipelines y registry, MySQL ≥5.x para el Model Registry y cert-manager para KServe. En 3.x solo se soporta el modo RawDeployment. Es el reemplazo más directo de AI Essentials sobre hardware arbitrario, con soporte.
- **NVIDIA AI Enterprise.** No es una plataforma MLOps, sino la capa de software soportado: NIM, GPU/Network Operator, NeMo y blueprints. Se puede montar encima de cualquiera de las dos opciones. Se licencia por GPU.
- **Canonical Charmed Kubeflow + Charmed MLflow.** Kubeflow upstream con soporte comercial y validación en GPUs.

Recomendaciones

Arquitectura de referencia propuesta (empresa colombiana, 100 % offline):

1. **Zona de tránsito (DMZ con internet controlado).** Un “build host” que descarga imágenes, charts, wheels, paquetes del SO y pesos de modelos. Ahí se escanean (Trivy), se firman (cosign) y se empaquetan en bundles versionados que se transfieren con medio controlado o diodo de datos.
2. **Servicios base dentro de la burbuja:** Harbor como registro OCI de imágenes, charts y modelos; devpi o Nexus para PyPI, y apt-mirror; Gitea/GitLab con Argo CD; Keycloak federado a AD; CA interna con cert-manager; **Ceph RGW (Rook) o SeaweedFS** para S3, en lugar de MinIO Community; **PostgreSQL** gestionado con un operador (por ejemplo CloudNativePG) para MLflow.
3. **Kubernetes. RKE2** (u **OpenShift** si se quiere comprar soporte) con 3 nodos de control y nodos GPU x86. **GPU Operator** con drivers precompilados, DCGM exporter, MIG o time-slicing según la carga, y **Kueue** para cuotas de GPU por equipo.
4. **Plataforma ML: Kubeflow Community Distribution 26.03.1**, pineada, con Profiles, Notebooks, KFP, Katib, Trainer y KServe; **MLflow 3.x** como servicio central (tracking, registry, tracing y evaluación GenAI), con PostgreSQL y S3 y proxy de artefactos, protegido por OAuth2-proxy/Keycloak; Kubeflow Hub desactivado o solo como catálogo.
5. **Serving: KServe en modo Standard** con Gateway API; runtime **vLLM** para LLMs (pesos en S3 o como imágenes OCI de modelo) y MLServer para modelos MLflow clásicos; `LLMInferenceService` solo para pilotos que necesiten prefill/decode o enrutamiento por KV-cache, por ser API alpha; delante, un **gateway compatible con OpenAI** (LiteLLM o Envoy AI Gateway) con claves por equipo, y Open WebUI para usuarios finales.
6. **Observabilidad y gobierno.** Prometheus, Grafana y Loki; trazas GenAI en MLflow; Kyverno; Velero; y un runbook de upgrade semestral alineado a las releases de Kubeflow (26.03 → 26.10).
7. **Ruta de adopción.**
   - Fase 0 (4–6 semanas): laboratorio en k3s sobre el DGX Spark/GX10, para validar flujos y la compatibilidad ARM64.

    - Fase 1: clúster RKE2 con MLflow + KServe/vLLM. Aporta el valor más inmediato porque ya dominas vLLM.
    - Fase 2: Kubeflow completo (Pipelines, Notebooks, multi-tenancy).
    - Fase 3: evaluar si el esfuerzo justifica soporte comercial (OpenShift AI o Canonical) en lugar de seguir DIY.

Decisión práctica:

- Si la empresa **va a comprar hardware HPE de todas formas**, pide la cotización de PCAI con AI Essentials y compárala contra el costo de 2–3 FTE. El software de HPE es esencialmente este mismo stack ya integrado.
- Si el hardware **es heterogéneo o ya existe**, el stack open source es un reemplazo funcionalmente completo, con OpenShift AI como plan B si se necesita un SLA.

Advertencias

- **Versiones de HPE.** Salen de extractos de la Support Matrix oficial (la página completa no se pudo leer íntegra). No se verificó la versión exacta de KServe ni de Istio dentro de AIE, solo el SDK `kserve==0.14.1` de los notebooks.
- **Precios.** HPE no publica precios ni la unidad exacta de la E-LTU de AI Essentials. Cualquier comparación de TCO requiere cotización local.
- **Estimaciones propias.** El esfuerzo de "2–3 FTE" y las duraciones de fase son juicios propios, no datos de una fuente.
- **Modo por defecto de KServe.** Las fuentes son contradictorias (la documentación oficial indica Knative; un tutorial sobre v0.20 indica Standard). Hay que fijarlo explícitamente.
- **APIs inmaduras.** `LLMInferenceService` y Kubeflow Workspaces (v2) son alpha y beta, con cambios incompatibles entre releases.
- **Air-gapped de HPE.** La disponibilidad citada proviene de comunicados de HPE (marzo y junio de 2026) y de las QuickSpecs. El anuncio de readiness IL4 corresponde a HPE Private Cloud PC7000, no necesariamente a PCAI.
- **Cambios por venir.** Kubeflow 26.10 y KServe 0.21 son inminentes. Revisa las notas de release antes de congelar el bundle offline.

Fuentes principales

- HPE AI Essentials — Support Matrix 2026.07: support.hpe.com (docId a00aie202607hen_us)
- HPE AI Essentials — Overview y documentación 1.8/1.12: support.hpe.com
- HPE Private Cloud AI — comunicado marzo 2026: hpe.com/us/en/newsroom
- HPE MLIS release notes: docs.ai-solutions.ext.hpe.com/products/mlis/release-notes
- HPE Store — AI Essentials 1-year E-LTU (S6E23AAE): buy.hpe.com
- Kubeflow 26.03 release: blog.kubeflow.org/kubeflow-26.03-release
- Kubeflow 1.11 release: blog.kubeflow.org/kubeflow-1.11-release
- kubeflow/community-distribution (README y releases): github.com/kubeflow/community-distribution

- CNCF — Kubeflow project page y blog julio 2026: cncf.io
- Instalación de Kubeflow: kubeflow.org/docs/started/installing-kubeflow
- MLflow releases y changelog: mlflow.org/releases; github.com/mlflow/mlflow
- MLflow tracking server architecture: mlflow.org/docs/latest/self-hosting
- MLflow → KServe tutorial: mlflow.org/docs/latest/deployment/deploy-model-to-kubernetes/tutorial
- KServe (repo, blog, admin guide, configurations, multi-node): github.com/kserve/kserve; kserve.github.io/website
- KServe discussion #4100: github.com/kserve/kserve/discussions/4100
- NVIDIA GPU Operator air-gapped install: docs.nvidia.com/datacenter/cloud-native/gpu-operator
- Red Hat OpenShift AI — MLflow (3.4), instalación desconectada, Helm/GitOps: docs.redhat.com; developers.redhat.com
- red-hat-data-services/mlflow-operator: github.com
- Canonical Charmed Kubeflow releases: github.com/canonical/charmed-kubeflow-solutions
- Alternativas a MinIO (S3 para lakehouse): iomete.com; community.getgrist.com